PEARL Notes
From S-AM to T-AM through T-Definitional Interpreter

Joongwon Ahn, Jay Lee

May 14, 2026

1 Overview

This document defines a revised A-normal form (ANF) language with:

- a finite set of global, mutually recursive, non-capturing function definitions,
- constructor values such as $\mathsf{some}(3)$ and $\mathsf{cons}(3, \mathsf{nil})$,
- first-order pattern matching on constructor tag and arity.

The semantics is given by a small-step abstract machine in the style of the CEK machine from Flanagan et al. [1]. As in that machine, evaluation proceeds with environments and continuations. Unlike the usual closure-based presentation, control is represented by labels and a static control map, function application uses global lookup instead of closures, and matching dispatches directly on constructor tag and arity. Because the language is pure and constructor values are finite trees, we do not use store indirection: variables are bound directly to values in the environment.

2 Syntax

$$x \in \mathsf{Var} \qquad f \in \mathsf{Fun} \qquad n \in \mathsf{Int} \qquad t \in \mathsf{Tag} \qquad o \in \mathsf{Prim} \qquad \ell \in \mathsf{Label}$$

$$\begin{array}{llll}
\mathsf{Prog} & P & ::= & \overline{f(\overline{x}) = c}; c \\
\mathsf{Exp} & e & ::= & n \mid x \mid o(\overline{e}) \\
\mathsf{Cmd} & c & ::= & x \\
 & & \mid & \textbf{let } x = e \textbf{ in } c \\
 & & \mid & \textbf{let } x = t\langle \overline{x} \rangle \textbf{ in } c \\
 & & \mid & \textbf{let } x = f(\overline{x}) \textbf{ in } c \\
 & & \mid & \textbf{match } e \textbf{ with } \overline{b} \textbf{ end} \\
 & b & ::= & t\langle \overline{x} \rangle \Rightarrow c
\end{array}$$

We uniquely identify syntactically identical commands c in different occurrences by labeling them ${}^{\ell}c$, which we have omitted above for brevity. In the remainder of the paper, we usually omit explicit command labels and write c for ${}^{\ell}c$; by slight abuse of notation, we also use c to denote its associated label ℓ when no confusion arises.

3 CEK

Machine states σ combine the current control label ℓ, current environment ρ, and current continuation κ.

Values v are the data manipulated by the machine during execution. Since the language has neither mutation nor cyclic heap structure, recursive data can be represented directly as inductive values rather than indirectly through addresses and a store.

A continuation frame $\langle \ell, \rho \rangle$ records the suspended control label ℓ together with the environment ρ to restore when the current computation returns. A continuation is a stack of such frames, with the head of the list denoting the next frame to receive a returned value.

Traces π are finite sequences of states σ.

$$\begin{array}{rcl}
\mathsf{State} & \triangleq & \mathsf{Label} \times \mathsf{Env} \times \mathsf{Kont} \\
\mathsf{Value} & \triangleq & \mathsf{Int} + \mathsf{Tag} \times \mathsf{Value}^* \\
\mathsf{Env} & \triangleq & \mathsf{Var} \rightharpoonup \mathsf{Value} \\
\mathsf{Kont} & \triangleq & (\mathsf{Label} \times \mathsf{Env})^* \\
\mathsf{Trace} & \triangleq & \mathsf{State}^*
\end{array}$$

$$\begin{array}{rrcl}
\mathsf{State} & \sigma & ::= & \langle \ell, \rho, \kappa \rangle \\
\mathsf{Value} & v & ::= & n \mid t\langle \overline{v} \rangle \\
\mathsf{Env} & \rho & ::= & \{\overline{x \mapsto v}\} \\
\mathsf{Kont} & \kappa & ::= & [\overline{\langle \ell, \rho \rangle}] \\
\mathsf{Trace} & \pi & ::= & [\overline{\sigma}]
\end{array}$$

We represent Booleans via $\texttt{true} \triangleq \texttt{true}\langle\rangle$ and $\texttt{false} \triangleq \texttt{false}\langle\rangle$.

4 Operational Semantics

4.1 Evaluation of Expressions

The interpretation of expressions is the partial function $\mathcal{E}[\![\cdot]\!] \colon \mathsf{Exp} \to \mathsf{Env} \rightharpoonup \mathsf{Value}$ defined by

$$\begin{aligned}
\mathcal{E}[\![n]\!]\,\rho &\triangleq n \\
\mathcal{E}[\![x]\!]\,\rho &\triangleq \rho(x) \\
\mathcal{E}[\![o(\overline{e_i})]\!]\,\rho &\triangleq \mathcal{O}[\![o]\!](\overline{v_i}) && \text{where } v_i = \mathcal{E}[\![e_i]\!]\,\rho
\end{aligned}$$

where $\mathcal{O}[\![\cdot]\!] \colon \mathsf{Prim} \to \mathsf{Value}^* \rightharpoonup \mathsf{Value}$ is the interpretation of primitives o.

4.2 Transition Relation

The relation[1]

$$\sigma \overset{P}{\leadsto} \sigma'$$

means that program P takes one small-step from state σ to state σ'.

[1]We omit P in $\overset{P}{\leadsto}$ when obvious.

$$\begin{array}{lll} & \langle y, \rho, \langle \mathbf{let}\ x = f(\overline{z_i})\ \mathbf{in}\ c, \rho' \rangle :: \kappa \rangle & \\ \rightsquigarrow & \langle c, \rho'\{x \mapsto v\}, \kappa \rangle & [\textsc{Return}] \\ & \quad \text{where } v = \rho(y) & \end{array}$$

$$\begin{array}{lll} & \langle \mathbf{let}\ x = e\ \mathbf{in}\ c, \rho, \kappa \rangle & \\ \rightsquigarrow & \langle c, \rho\{x \mapsto v\}, \kappa \rangle & [\textsc{LetExp}] \\ & \quad \text{where } v = \mathcal{E}[\![e]\!]\,\rho & \end{array}$$

$$\begin{array}{lll} & \langle \mathbf{let}\ x = t\langle \overline{y_i} \rangle\ \mathbf{in}\ c, \rho, \kappa \rangle & \\ \rightsquigarrow & \langle c, \rho\{x \mapsto t\langle \overline{v_i} \rangle\}, \kappa \rangle & [\textsc{LetTag}] \\ & \quad \text{where } v_i = \rho(y_i) & \end{array}$$

$$\begin{array}{lll} & \langle \mathbf{let}\ x = f(\overline{y_i})\ \mathbf{in}\ c, \rho, \kappa \rangle & \\ \rightsquigarrow & \langle c_f, \{\overline{x_i \mapsto v_i}\}, \langle \mathbf{let}\ x = f(\overline{y_i})\ \mathbf{in}\ c, \rho \rangle :: \kappa \rangle & [\textsc{LetCall}] \\ & \quad \text{where } v_i = \rho(y_i),\ \big(f(\overline{x_i}) = c_f\big) \in P & \end{array}$$

$$\begin{array}{lll} & \langle \mathbf{match}\ e\ \mathbf{with}\ \overline{b}, \rho, \kappa \rangle & \\ \rightsquigarrow & \langle c, \rho\{\overline{x_i \mapsto v_i}\}, \kappa \rangle & [\textsc{Match}] \\ & \quad \text{where } \mathcal{E}[\![e]\!]\,\rho = t\langle \overline{v_i} \rangle,\ \big(t\langle \overline{x_i} \rangle \Rightarrow c\big) \in \overline{b} & \end{array}$$

Note that in the above rules, each label is denoted by its associated command.

The Return rule only needs to inspect suspended call sites: LetCall is the only rule that extends the continuation, so every nonempty continuation is headed by a label whose node is a function-binding let. Thus the variable to bind is recovered by inspecting the node associated with the suspended label. In contrast to the CESK presentation, no fresh address allocation is needed: each binding in the rules above extends the environment directly with the computed value.

4.3 Initial Configuration

Evaluation of the program $P = \overline{f(\overline{x}) = c}; c_0$ with initial environment ρ starts from

$$\mathsf{inj}(P, \rho) \triangleq \langle c_0, \rho, [] \rangle.$$

The initial environment ρ supplies the values of the free variables of the main term. It can be viewed as the input to the program. In the implementation, the parser accepts a list of function definitions and chooses a distinguished `main` function when one exists, otherwise the last definition is used as the entry point. This is equivalent to taking the body of that distinguished function as c_0 and the user-supplied environment as its initial environment.

4.4 Trace Semantics

Let $\overset{P}{\rightsquigarrow}{}^*$ be the reflexive-transitive closure of $\overset{P}{\rightsquigarrow}$. Because the main term is a command of the source language, termination occurs at a state of the form $\langle x, \rho, [] \rangle$ where x is the "return command" at the end of the main term. Such a state is final: it consists of the label of the last return command, an environment, and an empty continuation. No transition rule applies to such states.

The value semantics of programs is the partial function

$$\mathcal{V}[\![\cdot]\!] : \mathsf{Prog} \to \mathsf{Env} \rightharpoonup \mathsf{Value}$$

defined by

$$\mathcal{V}[\![P]\!]\,\rho \triangleq \rho'(x) \qquad \text{where } \mathsf{inj}(P, \rho) \overset{P}{\rightsquigarrow}{}^* \langle x, \rho', [] \rangle.$$

The value semantic function is partial. Evaluation may fail to produce a value because execution gets stuck, for example because of an undefined variable or function lookup, an undefined primitive application, or a failed match, or because of an infinite transition sequence.

The trace semantics of programs is the function

$$\mathcal{T}[\![\cdot]\!] : \mathsf{Prog} \to \mathsf{Env} \to \wp(\mathsf{Trace})$$

defined by

$$\mathcal{T}[\![P]\!]\,\rho \triangleq \left\{ \langle \sigma_0, \ldots, \sigma_n \rangle \mid \sigma_0 = \mathsf{inj}(P, \rho) \land \forall 0 \leq i < n,\ \sigma_i \overset{P}{\rightsquigarrow} \sigma_{i+1} \right\}.$$

Thus $\mathcal{T}[\![P]\!]\,\rho$ is the set of all finite prefixes of transition sequences starting from $\mathsf{inj}(P, \rho)$. The trace semantic function is total; it always returns a set of traces, including traces that end in a stuck state or prefixes of a diverging execution.

5 T Definitional Interpreter

```
# prog ::= Prog(defs, exp)
# defs ::= Defs(Fun(int), exp, defs) | Nil()
# env  ::= Env(Var(int, int), int, env) | Nil()
# exp  ::= Int(int, int) | Var(int, int) | Sub(int, exp, exp) | Mul(int, exp, ex
#        | Let(int, Var(int, int), exp, exp) | App(int, Fun(int), exp) | Ifz(int

fundef(defs, fid) =
  match defs with
  | Defs(f, body, rest) =>
    match f with
    | Fun(fid2) =>
      match iszero(sub(fid, fid2)) with
      | True() => body
      | False() => let r = fundef(rest, fid) in r
      end
    end
  end

lookup(env, xid) =
  match env with
  | Env(x, val, rest) =>
    match x with
    | Var(l, xid2) =>
      match iszero(sub(xid, xid2)) with
      | True() => val
      | False() => let r = lookup(rest, xid) in r
      end
    end
  end

extend(env, x, val) =
  let r = Env(x, val, env) in r

eval(e, env, defs) =
  ℓevalmatch e with
```

```
  | Int(l, n) => ℓ_intr n
  | Var(l, xid) =>
    let v = lookup(env, xid) in ℓ_varr v
  | Sub(l, e1, e2) =>
    ℓ_sub1 let v1 = eval(e1, env, defs) in
    ℓ_sub2 let v2 = eval(e2, env, defs) in
    let r = sub(v1, v2) in ℓ_subr r
  | Mul(l, e1, e2) =>
    ℓ_mul1 let v1 = eval(e1, env, defs) in
    ℓ_mul2 let v2 = eval(e2, env, defs) in
    let r = mul(v1, v2) in ℓ_mulr r
  | Let(l, x, e1, e2) =>
    ℓ_let1 let v1 = eval(e1, env, defs) in
    let new_env = extend(env, x, v1) in
    ℓ_let2 let r = eval(e2, new_env, defs) in ℓ_letr r
  | App(l, f, e1) =>
    match f with
    | Fun(fid) =>
      ℓ_app1 let v = eval(e1, env, defs) in
      let body = fundef(defs, fid) in
      let empty_env = Nil() in
      let l0 = 0 in
      let x0 = 0 in
      let x = Var(l0, x0) in # function parameter as xid 0
      let call_env = extend(empty_env, x, v) in
      ℓ_app2 let r = eval(body, call_env, defs) in ℓ_appr r
    end
  | Ifz(l, e1, e2, e3) =>
    ℓ_ifz1 let v1 = eval(e1, env, defs) in
    match iszero(v1) with
    | True() => ℓ_ifz2 let r = eval(e2, env, defs) in ℓ_ifz2r r
    | False() => ℓ_ifz3 let r = eval(e3, env, defs) in ℓ_ifz3r r
    end
  end

main(p, arg) =
  match p with
  | Prog(defs, e) =>
    let empty_env = Nil() in
    let l0 = 0 in
    let x0 = 0 in
    let x = Var(l0, x0) in
    let initial_env = extend(empty_env, x, arg) in
    ℓ_maineval let r = eval(e, initial_env, defs) in r
  end
```

Interpreter labels. The following concrete labels are the current parser labels for the S definitional interpreter. The extraction uses the symbolic labels; the numeric labels are only for matching raw S-CEK traces.

Symbol	Interpreter point	Label
ℓ_{eval}	`eval`: dispatch match on `e`	14
ℓ_{intr}	`Int`: return `n`	15
ℓ_{varr}	`Var`: return `v`	17
ℓ_{subr}	`Sub`: return `r` after `sub(v1,v2)`	21
ℓ_{sub1}	`Sub`: first recursive `eval(e1, env, defs)`	18
ℓ_{sub2}	`Sub`: second recursive `eval(e2, env, defs)`	19
ℓ_{mulr}	`Mul`: return `r` after `mul(v1,v2)`	25
ℓ_{mul1}	`Mul`: first recursive `eval(e1, env, defs)`	22
ℓ_{mul2}	`Mul`: second recursive `eval(e2, env, defs)`	23
ℓ_{letr}	`Let`: return `r`	29
ℓ_{let1}	`Let`: first recursive `eval(e1, env, defs)`	26
ℓ_{let2}	`Let`: body recursive `eval(e2, new_env, defs)`	28
ℓ_{appr}	`App`: return `r`	39
ℓ_{app1}	`App`: argument recursive `eval(e1, env, defs)`	31
ℓ_{app2}	`App`: body recursive `eval(body, call_env, defs)`	38
ℓ_{ifz2r}	`Ifz`: then-branch return `r`	43
ℓ_{ifz3r}	`Ifz`: else-branch return `r`	45
ℓ_{ifz1}	`Ifz`: test recursive `eval(e1, env, defs)`	40
ℓ_{ifz2}	`Ifz`: then recursive `eval(e2, env, defs)`	42
ℓ_{ifz3}	`Ifz`: else recursive `eval(e3, env, defs)`	44
$\ell_{maineval}$	`main`: final recursive `eval(e, initial_env, defs)`	52

6 Extracting a T Abstract Machine

$$\lfloor\cdot\rfloor : \mathsf{Value}_S \rightharpoonup \mathsf{Exp}_T + \mathsf{Defs}_T + \mathsf{Env}_T$$

$$\begin{aligned}
\lfloor \mathtt{Int}\langle \ell, n\rangle \rfloor &\triangleq {}^{\ell}n \\
\lfloor \mathtt{Var}\langle \ell, x\rangle \rfloor &\triangleq {}^{\ell}x \\
\lfloor \mathtt{Sub}\langle \ell, e_1, e_2\rangle \rfloor &\triangleq {}^{\ell}(\lfloor e_1\rfloor \texttt{-} \lfloor e_2\rfloor) \\
\lfloor \mathtt{Mul}\langle \ell, e_1, e_2\rangle \rfloor &\triangleq {}^{\ell}(\lfloor e_1\rfloor \texttt{*} \lfloor e_2\rfloor) \\
\lfloor \mathtt{Let}\langle \ell, x, e_1, e_2\rangle \rfloor &\triangleq {}^{\ell}(\mathsf{let}\ \lfloor x\rfloor_{\mathsf{var}} = \lfloor e_1\rfloor\ \mathsf{in}\ \lfloor e_2\rfloor) \\
\lfloor \mathtt{App}\langle \ell, f, e\rangle \rfloor &\triangleq {}^{\ell}(\lfloor f\rfloor_{\mathsf{fun}}(\lfloor e\rfloor)) \\
\lfloor \mathtt{Ifz}\langle \ell, e_0, e_1, e_2\rangle \rfloor &\triangleq {}^{\ell}(\mathsf{ifz}\ \lfloor e_0\rfloor\ \mathsf{then}\ \lfloor e_1\rfloor\ \mathsf{else}\ \lfloor e_2\rfloor) \\
\lfloor \mathtt{Nil}\langle\rangle \rfloor &\triangleq \varnothing \\
\lfloor \mathtt{Defs}\langle \mathsf{Fun}\langle f\rangle, e, l\rangle \rfloor &\triangleq \lfloor l\rfloor\{f \mapsto \lfloor e\rfloor\} \\
\lfloor \mathtt{Env}\langle \mathtt{Var}\langle \ell, x\rangle, n, l\rangle \rfloor &\triangleq \lfloor l\rfloor\{x \mapsto n\}
\end{aligned}$$

The last clause is defined only when the environment payload n is an encoded integer. where helper functions are

$$\begin{aligned}
\lfloor \mathtt{Var}\langle \ell, x\rangle \rfloor_{\mathsf{var}} &\triangleq x \\
\lfloor \mathsf{Fun}\langle f\rangle \rfloor_{\mathsf{fun}} &\triangleq f
\end{aligned}$$

We can define decoded T-state σ_T as

$$\begin{array}{llll} \mathsf{State}_T & \sigma_T & ::= & \langle q_T, \rho_T, \kappa_T \rangle \\ & q_T & ::= & e_T \mid n \\ \mathsf{Var}_T & x & ::= & n \\ \mathsf{Env}_T & \rho_T & ::= & \{\overline{x \mapsto n}\} \end{array}$$

Given an initial S-environment

$$\rho_S^0 = \{\mathsf{p} \mapsto \mathtt{Prog}\langle \mathit{defs}, e\rangle, \mathtt{arg} \mapsto n\},$$

we can decode a T-definition table

$$D_T \triangleq \lfloor \mathit{defs} \rfloor$$

and a T-initial state

$$\sigma_T^0 \triangleq \langle \lfloor e \rfloor, \{0 \mapsto n\}, []\rangle.$$

We now define state and continutation decoding,

$$\lfloor \cdot \rfloor : \mathsf{State}_S \rightharpoonup \mathsf{State}_T, \qquad \lfloor \cdot \rfloor : \mathsf{Kont}_S \to \mathsf{Kont}_T$$

as

$$\lfloor \langle \ell_{\mathit{eval}}, \rho_S, \kappa_S \rangle \rfloor \triangleq \langle \lfloor \rho_S(\mathsf{e}) \rfloor, \lfloor \rho_S(\mathsf{env}) \rfloor, \lfloor \kappa_S \rfloor \rangle$$

$$\lfloor \langle \ell, \rho_S, \kappa_S \rangle \rfloor \triangleq \begin{cases} \langle \mathsf{ret}_\ell(\rho_S), \lfloor \rho_S(\mathsf{env}) \rfloor, \lfloor \kappa_S \rfloor \rangle & \text{if } \mathsf{ret}_\ell(\rho_S) \neq \bot \\ \bot & \text{otherwise,} \end{cases}$$

where

$$\begin{aligned} \mathsf{ret}_{\ell_{\mathit{intr}}}(\rho_S) &\triangleq \rho_S(\mathsf{n}) \\ \mathsf{ret}_{\ell_{\mathit{varr}}}(\rho_S) &\triangleq \rho_S(\mathsf{v}) \\ \mathsf{ret}_{\ell_{\mathit{subr}}}(\rho_S) &\triangleq \rho_S(\mathsf{r}) \\ \mathsf{ret}_{\ell_{\mathit{mulr}}}(\rho_S) &\triangleq \rho_S(\mathsf{r}) \\ \mathsf{ret}_{\ell_{\mathit{letr}}}(\rho_S) &\triangleq \rho_S(\mathsf{r}) \\ \mathsf{ret}_{\ell_{\mathit{appr}}}(\rho_S) &\triangleq \rho_S(\mathsf{r}) \\ \mathsf{ret}_{\ell_{\mathit{ifz2r}}}(\rho_S) &\triangleq \rho_S(\mathsf{r}) \\ \mathsf{ret}_{\ell_{\mathit{ifz3r}}}(\rho_S) &\triangleq \rho_S(\mathsf{r}) \\ \mathsf{ret}_{\ell}(\rho_S) &\triangleq \bot \qquad \text{otherwise,} \end{aligned}$$

and

$$\begin{aligned} \lfloor [] \rfloor &\triangleq [] \\ \lfloor \langle \ell_{\mathit{sub1}}, \rho_S \rangle :: \kappa_S \rfloor &\triangleq \mathsf{Sub1}\langle \lfloor \rho_S(\mathsf{e2}) \rfloor, \lfloor \rho_S(\mathsf{env}) \rfloor \rangle :: \lfloor \kappa_S \rfloor \\ \lfloor \langle \ell_{\mathit{sub2}}, \rho_S \rangle :: \kappa_S \rfloor &\triangleq \mathsf{Sub2}\langle \rho_S(\mathsf{v1}), \lfloor \rho_S(\mathsf{env}) \rfloor \rangle :: \lfloor \kappa_S \rfloor \\ \lfloor \langle \ell_{\mathit{mul1}}, \rho_S \rangle :: \kappa_S \rfloor &\triangleq \mathsf{Mul1}\langle \lfloor \rho_S(\mathsf{e2}) \rfloor, \lfloor \rho_S(\mathsf{env}) \rfloor \rangle :: \lfloor \kappa_S \rfloor \\ \lfloor \langle \ell_{\mathit{mul2}}, \rho_S \rangle :: \kappa_S \rfloor &\triangleq \mathsf{Mul2}\langle \rho_S(\mathsf{v1}), \lfloor \rho_S(\mathsf{env}) \rfloor \rangle :: \lfloor \kappa_S \rfloor \\ \lfloor \langle \ell_{\mathit{let1}}, \rho_S \rangle :: \kappa_S \rfloor &\triangleq \mathsf{Let}\langle \lfloor \rho_S(\mathsf{x}) \rfloor_{\mathsf{var}}, \lfloor \rho_S(\mathsf{e2}) \rfloor, \lfloor \rho_S(\mathsf{env}) \rfloor \rangle :: \lfloor \kappa_S \rfloor \\ \lfloor \langle \ell_{\mathit{let2}}, \rho_S \rangle :: \kappa_S \rfloor &\triangleq \mathsf{Restore}\langle \lfloor \rho_S(\mathsf{env}) \rfloor \rangle :: \lfloor \kappa_S \rfloor \\ \lfloor \langle \ell_{\mathit{app1}}, \rho_S \rangle :: \kappa_S \rfloor &\triangleq \mathsf{App}\langle \lfloor \rho_S(\mathsf{f}) \rfloor_{\mathsf{fun}}, \lfloor \rho_S(\mathsf{env}) \rfloor \rangle :: \lfloor \kappa_S \rfloor \\ \lfloor \langle \ell_{\mathit{app2}}, \rho_S \rangle :: \kappa_S \rfloor &\triangleq \mathsf{Restore}\langle \lfloor \rho_S(\mathsf{env}) \rfloor \rangle :: \lfloor \kappa_S \rfloor \\ \lfloor \langle \ell_{\mathit{ifz1}}, \rho_S \rangle :: \kappa_S \rfloor &\triangleq \mathsf{Ifz}\langle \lfloor \rho_S(\mathsf{e2}) \rfloor, \lfloor \rho_S(\mathsf{e3}) \rfloor, \lfloor \rho_S(\mathsf{env}) \rfloor \rangle :: \lfloor \kappa_S \rfloor \\ \lfloor \langle \ell, \rho_S \rangle :: \kappa_S \rfloor &\triangleq \lfloor \kappa_S \rfloor \qquad \text{otherwise.} \end{aligned}$$

Then we have T-transition relation

$$
\begin{array}{lrcl}
\textsc{Int} & \langle {}^{\ell}n, \rho, \kappa \rangle & \rightsquigarrow & \langle n, \rho, \kappa \rangle \\
\textsc{Var} & \langle {}^{\ell}x, \rho, \kappa \rangle & \rightsquigarrow & \langle \rho(x), \rho, \kappa \rangle \\
\textsc{Sub1} & \langle {}^{\ell}(e_1\text{-}e_2), \rho, \kappa \rangle & \rightsquigarrow & \langle e_1, \rho, \mathsf{Sub1}\langle e_2, \rho \rangle :: \kappa \rangle \\
\textsc{Sub2} & \langle n_1, \rho', \mathsf{Sub1}\langle e_2, \rho \rangle :: \kappa \rangle & \rightsquigarrow & \langle e_2, \rho, \mathsf{Sub2}\langle n_1, \rho \rangle :: \kappa \rangle \\
\textsc{SubR} & \langle n_2, \rho', \mathsf{Sub2}\langle n_1, \rho \rangle :: \kappa \rangle & \rightsquigarrow & \langle n_1\text{-}n_2, \rho, \kappa \rangle \\
\textsc{Mul1} & \langle {}^{\ell}(e_1 * e_2), \rho, \kappa \rangle & \rightsquigarrow & \langle e_1, \rho, \mathsf{Mul1}\langle e_2, \rho \rangle :: \kappa \rangle \\
\textsc{Mul2} & \langle n_1, \rho', \mathsf{Mul1}\langle e_2, \rho \rangle :: \kappa \rangle & \rightsquigarrow & \langle e_2, \rho, \mathsf{Mul2}\langle n_1, \rho \rangle :: \kappa \rangle \\
\textsc{MulR} & \langle n_2, \rho', \mathsf{Mul2}\langle n_1, \rho \rangle :: \kappa \rangle & \rightsquigarrow & \langle n_1 * n_2, \rho, \kappa \rangle \\
\textsc{Let1} & \langle {}^{\ell}(\mathsf{let}\ x = e_1\ \mathsf{in}\ e_2), \rho, \kappa \rangle & \rightsquigarrow & \langle e_1, \rho, \mathsf{Let}\langle x, e_2, \rho \rangle :: \kappa \rangle \\
\textsc{Let2} & \langle n, \rho', \mathsf{Let}\langle x, e_2, \rho \rangle :: \kappa \rangle & \rightsquigarrow & \langle e_2, \rho\{x \mapsto n\}, \mathsf{Restore}\langle \rho \rangle :: \kappa \rangle \\
\textsc{Restore} & \langle n, \rho', \mathsf{Restore}\langle \rho \rangle :: \kappa \rangle & \rightsquigarrow & \langle n, \rho, \kappa \rangle \\
\textsc{App1} & \langle {}^{\ell}(f(e)), \rho, \kappa \rangle & \rightsquigarrow & \langle e, \rho, \mathsf{App}\langle f, \rho \rangle :: \kappa \rangle \\
\textsc{App2} & \langle n, \rho', \mathsf{App}\langle f, \rho \rangle :: \kappa \rangle & \rightsquigarrow & \langle D_T(f), \{0 \mapsto n\}, \mathsf{Restore}\langle \rho \rangle :: \kappa \rangle \\
\textsc{Ifz1} & \langle {}^{\ell}(\mathsf{ifz}\ e_1\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3), \rho, \kappa \rangle & \rightsquigarrow & \langle e_1, \rho, \mathsf{Ifz}\langle e_2, e_3, \rho \rangle :: \kappa \rangle \\
\textsc{Ifz2} & \langle 0, \rho', \mathsf{Ifz}\langle e_2, e_3, \rho \rangle :: \kappa \rangle & \rightsquigarrow & \langle e_2, \rho, \kappa \rangle \\
\textsc{Ifz3} & \langle n, \rho', \mathsf{Ifz}\langle e_2, e_3, \rho \rangle :: \kappa \rangle & \rightsquigarrow & \langle e_3, \rho, \kappa \rangle \qquad (n \neq 0)
\end{array}
$$

We define the compressed projection of an S-trace

$$\pi_S = \sigma_S^0, \ldots, \sigma_S^m$$

by

$$\mathsf{compress}(\pi_S) \triangleq \text{the sequence obtained by mapping each } \sigma_S^i \text{ through } \lfloor \cdot \rfloor,$$

dropping undefined projections, dropping projections whose control is $\perp$, and dropping adjacent duplicate T-states.

Theorem 1 (Projection simulation). *Let π_S be a finite S-CEK trace prefix produced by running the S definitional interpreter on an initial S-environment*

$$\rho_S^0 = \{\mathsf{p} \mapsto \mathtt{Prog}\langle \mathit{defs}, e \rangle,\ \mathtt{arg} \mapsto n\}.$$

Assume that $\lfloor \mathit{defs} \rfloor$ and $\lfloor e \rfloor$ are defined, and let

$$D_T \triangleq \lfloor \mathit{defs} \rfloor \qquad \sigma_T^0 \triangleq \langle \lfloor e \rfloor, \{0 \mapsto n\}, [] \rangle.$$

Assume also that every reached T variable is bound in its decoded environment, every reached function identifier is bound in D_T, every reached T expression evaluation produces an integer, and the S-CEK run does not get stuck. If

$$\pi_T \triangleq \mathsf{compress}(\pi_S),$$

then π_T is a finite prefix of the T-transition trace generated from σ_T^0. That is, if

$$\pi_T = \epsilon \quad \textit{or} \quad \pi_T = \sigma_T^0, \sigma_T^1, \ldots, \sigma_T^k,$$

then in the nonempty case, for every $0 \leq i < k$,

$$\sigma_T^i \rightsquigarrow \sigma_T^{i+1}.$$

Moreover, if π_S terminates with final integer value n', then π_T ends in a final T-state

$$\langle n', \rho_T, [] \rangle$$

for some decoded T-environment ρ_T.

Proof. Let a visible S-state be one whose label is either ℓ_{eval} or one of ℓ_{intr}, ℓ_{varr}, ℓ_{subr}, ℓ_{mulr}, ℓ_{letr}, ℓ_{appr}, ℓ_{ifz2r}, ℓ_{ifz3r}. All other interpreter states are administrative: they perform dispatch, `lookup`, `fundef`, `extend`, constructor allocation, or the S-level call/return plumbing needed to enter the next recursive `eval`. The function compress keeps exactly the decoded visible states, modulo adjacent duplicates.

We first record the decoding invariants used in every case. If $\lfloor e_S \rfloor = e_T$, then each encoded T subexpression of e_S decodes to the corresponding subexpression of e_T; this follows immediately by structural induction on the constructors `Int`, `Var`, Sub, `Mul`, `Let`, App, and `Ifz`. If $\lfloor \mathit{defs}_S \rfloor = D_T$, then a successful run of `fundef(defs, fid)` returns an encoded body b_S with $D_T(\mathit{fid}) = \lfloor b_S \rfloor$; this is induction on the `Defs`/`Nil` list and the interpreter's `iszero(sub(fid,fid2))` test. If $\lfloor \mathit{env}_S \rfloor = \rho_T$, then a successful run of `lookup(env, xid)` returns the integer $\rho_T(\mathit{xid})$; this is the analogous induction on the Env/Nil list. Finally, `extend(env, Var(_,x), n)` decodes to $\rho_T\{x \mapsto n\}$.

The continuation decoding is also invariant under administrative S-steps. The only S frames that decode to T frames are the suspended recursive `eval` call sites: ℓ_{sub1}, ℓ_{sub2}, ℓ_{mul1}, ℓ_{mul2}, ℓ_{let1}, ℓ_{let2}, ℓ_{app1}, ℓ_{app2}, and ℓ_{ifz1}. Their saved S environments contain exactly the payloads shown in the definition of $\lfloor \kappa_S \rfloor$, so decoding the S continuation gives the T continuation that the corresponding T transition pushes or expects. Frames for `lookup`, `fundef`, `extend`, branch calls, and constructor setup decode to no T frame and are therefore erased by compress.

Now consider a maximal S execution segment that starts at a visible state, contains no intermediate visible state with a distinct projection, and ends at the next kept visible state. We show that its two projections are related by one T step. The proof is by cases on the decoded control of the first visible state. For ${}^{\ell}n$ and ${}^{\ell}x$, the interpreter selects the `Int` or `Var` branch; the latter uses the lookup lemma. The next kept state is $\langle n, \rho_T, \kappa_T \rangle$ or $\langle \rho_T(x), \rho_T, \kappa_T \rangle$, respectively. For Sub and `Mul`, the first recursive `eval` call pushes Sub1 or Mul1, the second call replaces it by Sub2 or Mul2, and the primitive result state returns the arithmetic integer. These are exactly rules Sub1, Sub2, SubR, Mul1, Mul2, and MulR. For `Let`, the first recursive call pushes Let; the successful `extend` administrative segment decodes the body environment to $\rho_T\{x \mapsto n\}$ and pushes Restore, and the later return under ℓ_{let2} performs the Restore step. For App, the argument call pushes App; `fundef` supplies $D_T(f)$, constructor setup creates $\{0 \mapsto n\}$, and the body call pushes Restore, matching App1 and App2. For `Ifz`, the test call pushes Ifz, and the following `iszero` dispatch chooses the decoded then expression when the returned value is 0 and the else expression otherwise, matching Ifz1, Ifz2, and Ifz3. These cases exhaust the T controls reachable from decoded interpreter states.

The theorem follows by induction on adjacent pairs in compress(π_S). The empty projection case is immediate. In the nonempty case, the first kept state is the first recursive `eval` of the decoded program body from the decoded initial environment, so it is σ_T^0. Each induction step applies the local simulation argument above. If the S trace terminates with final integer value n', the last visible S return has empty decoded continuation and decoded control n', hence the final projected T-state is $\langle n', \rho_T, [] \rangle$ for its decoded environment ρ_T. □

Worked example: factorial at $n = 1$. Use the encoded T program

$$\texttt{Prog}\langle \texttt{Defs}\langle \texttt{Fun}\langle 0 \rangle, e_f, \texttt{Nil}\langle\rangle \rangle, e_0 \rangle$$

where

$$x_0 \triangleq 0, \qquad f_0 \triangleq 0, \qquad \rho_1 \triangleq \{0 \mapsto 1\}, \qquad \rho_0 \triangleq \{0 \mapsto 0\}$$
$$d \triangleq {}^{16}({}^{17}x_0\text{-}\,{}^{18}1), \qquad a \triangleq {}^{15}(f_0(d))$$
$$b \triangleq {}^{13}({}^{14}x_0 \star a), \qquad c \triangleq {}^{10}(\text{ifz } {}^{11}x_0 \text{ then } {}^{12}1 \text{ else } b)$$
$$e_f \triangleq c, \qquad e_0 \triangleq {}^{20}(f_0({}^{21}x_0))$$

Thus

$$D_T = \{0 \mapsto e_f\} \qquad \sigma_T^0 = \langle e_0, \rho_1, [] \rangle.$$

The raw S-CEK run for this input has

```
states=103    steps=102    end=final value=1.
```

In the projected table below, labelled numerals such as ${}^{12}1$ are T expressions, while bare numerals such as 1 are returned values.

#	S-state	q_T	ρ_T	κ_T
0	S_9	e_0	ρ_1	$[]$
1	S_{12}	${}^{21}x_0$	ρ_1	$\mathsf{App}\langle f_0, \rho_1\rangle$
2	S_{18}	1	ρ_1	$\mathsf{App}\langle f_0, \rho_1\rangle$
3	S_{32}	c	ρ_1	$\mathsf{Restore}\langle \rho_1\rangle$
4	S_{34}	${}^{11}x_0$	ρ_1	$\mathsf{Ifz}\langle {}^{12}1, b, \rho_1\rangle :: \mathsf{Restore}\langle \rho_1\rangle$
5	S_{40}	1	ρ_1	$\mathsf{Ifz}\langle {}^{12}1, b, \rho_1\rangle :: \mathsf{Restore}\langle \rho_1\rangle$
6	S_{43}	b	ρ_1	$\mathsf{Restore}\langle \rho_1\rangle$
7	S_{45}	${}^{14}x_0$	ρ_1	$\mathsf{Mul1}\langle a, \rho_1\rangle :: \mathsf{Restore}\langle \rho_1\rangle$
8	S_{51}	1	ρ_1	$\mathsf{Mul1}\langle a, \rho_1\rangle :: \mathsf{Restore}\langle \rho_1\rangle$
9	S_{53}	a	ρ_1	$\mathsf{Mul2}\langle 1, \rho_1\rangle :: \mathsf{Restore}\langle \rho_1\rangle$
10	S_{56}	d	ρ_1	$\mathsf{App}\langle f_0, \rho_1\rangle :: \mathsf{Mul2}\langle 1, \rho_1\rangle :: \mathsf{Restore}\langle \rho_1\rangle$
11	S_{58}	${}^{17}x_0$	ρ_1	$\mathsf{Sub1}\langle {}^{18}1, \rho_1\rangle :: \mathsf{App}\langle f_0, \rho_1\rangle :: \mathsf{Mul2}\langle 1, \rho_1\rangle :: \mathsf{Restore}\langle \rho_1\rangle$
12	S_{64}	1	ρ_1	$\mathsf{Sub1}\langle {}^{18}1, \rho_1\rangle :: \mathsf{App}\langle f_0, \rho_1\rangle :: \mathsf{Mul2}\langle 1, \rho_1\rangle :: \mathsf{Restore}\langle \rho_1\rangle$
13	S_{66}	${}^{18}1$	ρ_1	$\mathsf{Sub2}\langle 1, \rho_1\rangle :: \mathsf{App}\langle f_0, \rho_1\rangle :: \mathsf{Mul2}\langle 1, \rho_1\rangle :: \mathsf{Restore}\langle \rho_1\rangle$
14	S_{67}	1	ρ_1	$\mathsf{Sub2}\langle 1, \rho_1\rangle :: \mathsf{App}\langle f_0, \rho_1\rangle :: \mathsf{Mul2}\langle 1, \rho_1\rangle :: \mathsf{Restore}\langle \rho_1\rangle$
15	S_{69}	0	ρ_1	$\mathsf{App}\langle f_0, \rho_1\rangle :: \mathsf{Mul2}\langle 1, \rho_1\rangle :: \mathsf{Restore}\langle \rho_1\rangle$
16	S_{83}	c	ρ_0	$\mathsf{Restore}\langle \rho_1\rangle :: \mathsf{Mul2}\langle 1, \rho_1\rangle :: \mathsf{Restore}\langle \rho_1\rangle$
17	S_{85}	${}^{11}x_0$	ρ_0	$\mathsf{Ifz}\langle {}^{12}1, b, \rho_0\rangle :: \mathsf{Restore}\langle \rho_1\rangle :: \mathsf{Mul2}\langle 1, \rho_1\rangle :: \mathsf{Restore}\langle \rho_1\rangle$
18	S_{91}	0	ρ_0	$\mathsf{Ifz}\langle {}^{12}1, b, \rho_0\rangle :: \mathsf{Restore}\langle \rho_1\rangle :: \mathsf{Mul2}\langle 1, \rho_1\rangle :: \mathsf{Restore}\langle \rho_1\rangle$
19	S_{94}	${}^{12}1$	ρ_0	$\mathsf{Restore}\langle \rho_1\rangle :: \mathsf{Mul2}\langle 1, \rho_1\rangle :: \mathsf{Restore}\langle \rho_1\rangle$
20	S_{95}	1	ρ_0	$\mathsf{Restore}\langle \rho_1\rangle :: \mathsf{Mul2}\langle 1, \rho_1\rangle :: \mathsf{Restore}\langle \rho_1\rangle$
21	S_{97}	1	ρ_1	$\mathsf{Mul2}\langle 1, \rho_1\rangle :: \mathsf{Restore}\langle \rho_1\rangle$
22	S_{99}	1	ρ_1	$\mathsf{Restore}\langle \rho_1\rangle$
23	S_{101}	1	ρ_1	$[]$

Rows 0–2 evaluate the top-level application argument. Rows 3–6 enter the first factorial body under the top-level restore frame and select the nonzero branch. Rows 7–15 evaluate the multiplication's left operand and the recursive-call argument x_0-1. Rows 16–20 enter the recursive factorial body with ρ_0 and select the base case. Rows 21–23 restore the recursive caller environment, finish the pending multiplication, restore the top-level caller environment, and end in $\langle 1, \rho_1, []\rangle$.

References

[1] Cormac Flanagan, Amr Sabry, Bruce F. Duba, and Matthias Felleisen. 1993. The Essence of Compiling with Continuations. In *Proceedings of the ACM SIGPLAN 1993 Conference on Programming Language Design and Implementation* (Albuquerque, New Mexico, USA) *(PLDI '93)*. Association for Computing Machinery, New York, NY, USA, 237–247. doi:10.1145/155090.155113